

Resumo Completo — Computação Paralela e Distribuída (CPD)

IPS/EST Setúbal · LEI · 2025/2026

MÓDULO 1 — Introdução à Computação Paralela

1.1 Motivação: Fim da Lei de Moore

- **Lei de Moore (1965):** O número de transístores num chip duplica a cada 18–24 meses.
- **Power Wall (2004–2005):** Aumentar a frequência exige mais voltagem → mais calor → impossível de dissipar. Frequências estagnaram em ~3,0–4,5 GHz.
- **Solução:** Em vez de um núcleo mais rápido, integrar **múltiplos núcleos** no mesmo chip.
- **Consequência:** O desempenho escala pela **largura** (paralelismo), não pela profundidade (clock).

1.2 Computação Paralela — O Quê

Um sistema paralelo tem:

- **Múltiplos núcleos/processadores/máquinas** que trabalham em simultâneo.
- **Memória partilhada ou distribuída.**
- **Mecanismos de sincronização** para troca de dados.

Escala	Exemplo
Local	CPUs multi-core
Média	Clusters
Global	Supercomputadores (Exascale)

1.3 Execução Sequencial vs Paralela

- **Sequencial:** Tarefas executam uma a seguir à outra. Tempo total = T1 + T2 + T3 + T4.
- **Paralela:** Tarefas dividem-se por múltiplos núcleos. Tempo total reduzido.

1.4 Concorrência vs Paralelismo ⚠ (MUITO IMPORTANTE)

	Concorrência	Paralelismo
Definição	Gerir múltiplas tarefas ao mesmo tempo (alternando)	Executar múltiplas tarefas fisicamente ao mesmo tempo
Hardware	Funciona com 1 núcleo (time-sharing)	Requer múltiplos núcleos
Analogia	1 cozinheiro a alternar entre pratos	4 cozinheiros, cada um no seu prato

Frase-chave: "Concorrência é sobre *gerir* muitas coisas; Paralelismo é sobre *fazer* muitas coisas."

1.5 Medição de Desempenho: Speedup

$$S(p) = T_{\text{sequencial}} / T_{\text{paralelo}}$$

- **Speedup Linear (ideal):** $S(p) = p$ — dobro dos recursos = dobro da velocidade.
- **Speedup Sub-linear (real):** $S(p) < p$ — comunicação e sincronização limitam o ganho.

1.6 Lei de Amdahl

$$S(p) = 1 / ((1 - f) + f/p)$$

- f = fração paralelizável do código
- $(1-f)$ = fração sequencial (não paralelizável)
- Com processadores infinitos: speedup máximo = $1 / (1 - f)$

Implicações:

- Se 10% do código é sequencial → ganho máximo é **10x**, independentemente de quantos núcleos.
- A parte sequencial é o **gargalo** do sistema.
- Para ganhos reais: **reduzir a fração sequencial**.

1.7 Lei de Gustafson

$$S(p) = p - \alpha \times (p - 1)$$

- α = fração sequencial
- **Visão otimista:** com mais processadores, o utilizador escala o **tamanho do problema**, não apenas resolve o mesmo problema mais rápido.
- Amdahl: tamanho do problema fixo → speedup limitado.
- Gustafson: problema cresce com os recursos → speedup linear possível.

1.8 Tipos de Paralelismo

Paralelismo de Dados (Data Parallelism)

- A **mesma operação** aplicada a **diferentes partes dos dados** em simultâneo.
- Foco na partição dos dados.
- Usa **SIMD** (Single Instruction, Multiple Data).
- Comum em GPUs, Big Data, NumPy.
- Exemplo: vários pintores a pintar secções diferentes do mesmo muro com a mesma cor.

Paralelismo de Tarefas (Task Parallelism)

- **Diferentes operações/tarefas** executadas em paralelo (podem usar dados distintos).
- Foco na partição do código.
- Usa **MIMD** (Multiple Instruction, Multiple Data).
- Comum em sistemas multi-core.
- Exemplo: linha de montagem — motor, portas e pintura ao mesmo tempo.

1.9 Taxonomia de Flynn

Sigla	Nome	Descrição	Exemplo
SISD	Single Instruction Single Data	Execução sequencial clássica	CPU single-core
SIMD	Single Instruction Multiple Data	Mesma instrução, múltiplos dados	GPUs, NumPy
MIMD	Multiple Instruction Multiple Data	Processos independentes	<code>multiprocessing</code> Python

O módulo `multiprocessing` de Python segue o modelo **MIMD**.

1.10 CPU vs GPU

Característica	CPU	GPU
Filosofia	Latência (terminar uma tarefa rápido)	Débito (terminar muitas tarefas ao mesmo tempo)
Núcleos	Poucos (4–64), complexos e potentes	Milhares, mais simples
Otimizado para	Lógica complexa, SO, controlo de fluxo	Deep Learning, Gráficos, SIMD
Memória	Cache L3 generosa	Otimizada para dados massivos

1.11 Memória Partilhada vs Memória Distribuída

Memória Partilhada (Shared Memory)

- Todos os processadores acedem ao **mesmo espaço de memória**.
- Comunicação implícita (variáveis partilhadas).
- **Vantagens:** Programação mais simples, baixa latência.
- **Desvantagens:** Escalabilidade limitada, necessidade de locks, custo hardware.
- Exemplos Python: `threading.Thread`, `multiprocessing.shared_memory`.

Memória Distribuída (Distributed Memory)

- Cada nó tem a sua **própria memória local** — comunicação via rede.
- **Vantagens:** Escalabilidade quase ilimitada, sem contention, tolerância a falhas.
- **Desvantagens:** Programação mais complexa, latência de rede alta, serialização necessária.
- Ferramentas: MPI (`mpi4py`), Dask, Ray, Apache Spark.

1.12 Pipelining

- Várias tarefas executam **etapas diferentes em simultâneo** → maior throughput.
- Analogia: linha de montagem de automóveis — motor, portas e pintura em paralelo.
- As etapas comunicam via **Queues**.

MÓDULO 2 — Threads e Processos em Python

2.1 O GIL (Global Interpreter Lock) ⚠

- **GIL** = mecanismo do interpretador **CPython** que garante que apenas **uma thread** executa bytecode Python de cada vez.

- **Não impede concorrência** — impede **paralelismo em tarefas CPU-bound**.
- **O GIL é libertado** durante operações de I/O: pedidos de rede, leitura/escrita em disco, `time.sleep()`, consultas a bases de dados.
- Consequência: `threading` **não acelera tarefas CPU-bound** em CPython.

2.2 Threads em Python (`threading`)

O que é uma thread?

- Menor unidade de processamento gerida pelo SO dentro de um processo.
- Threads **partilham o mesmo espaço de memória** do processo pai.
- Comunicação rápida mas arriscada (race conditions).

Ciclo de vida de uma thread

New → Runnable → Running → Waiting → Terminated

API básica

```
import threading

def tarefa():
    # código aqui

t1 = threading.Thread(target=tarefa)
t1.start()    # inicia a execução
t1.join()    # bloqueia até terminar
```

Threads e GIL: I/O-bound vs CPU-bound

I/O-bound: GIL libertado durante espera → threads dão ganho real
CPU-bound: GIL retido → threads NÃO dão ganho

Demonstração prática:

- 2 threads × sleep(2s) → ~2s total (concorrência real, GIL libertado)
- 2 threads × cálculo intenso → ~mesmo tempo que 1 thread (GIL bloqueia)

2.3 Race Conditions

- Ocorrem quando múltiplas threads lêem/escrevem dados partilhados sem coordenação.
- `contador += 1` **não é atômica** (ler → modificar → escrever).
- O resultado final depende da **ordem de escalonamento** (não-determinístico).

```
# CÓDIGO COM RACE CONDITION
contador = 0
def incrementar():
    global contador
```

```
for _ in range(1000):  
    contador += 1 # não atômica!
```

2.4 Locks (`threading.Lock`)

- Garantem **exclusão mútua** na secção crítica.
- Apenas uma thread pode adquirir o lock de cada vez.
- Usar `with lock`: (prática recomendada — evita deadlocks por exceção).

```
lock = threading.Lock()  
def incrementar():  
    global contador  
    for _ in range(1000):  
        with lock:  
            contador += 1 # secção crítica protegida
```

2.5 CPU-bound vs I/O-bound

Tipo	Definição	Exemplos	Solução Python
CPU-bound	Tempo depende da velocidade do processador	Cálculo de primos, compressão, ML	<code>multiprocessing</code>
I/O-bound	Tempo à espera de recursos externos	Web scraping, leitura de ficheiros, APIs	<code>threading</code> ou <code>asyncio</code>

Regra de Ouro: "Se o CPU está a pensar, usa processos. Se está à espera, usa threads."

2.6 Processos em Python (`multiprocessing`)

O que é um processo?

- Unidade fundamental de isolamento no SO.
- Cada processo tem o **seu próprio espaço de memória**.
- Cada processo tem o **seu próprio GIL** → paralelismo real multi-core.

Vantagens:

- Isolamento de memória (erros num processo não afetam outros).
- Paralelismo real (contorna o GIL).

Desvantagens:

- Overhead de memória (cópia do estado do processo pai).
- Criação mais lenta que threads.
- Comunicação exige serialização (Pickle) + Pipes ou Queues.

Criação manual de processos

```
from multiprocessing import Process

def tarefa(n):
    return sum(i*i for i in range(n))

if __name__ == "__main__": # OBRIGATÓRIO em Windows/macOS
    p1 = Process(target=tarefa, args=(10**8,))
    p1.start()
    p1.join()
```

`if __name__ == "__main__":` é obrigatório para evitar criação recursiva de subprocessos.

Pool de Processos

- Abstração de alto nível: reservatório de trabalhadores prontos.
- `pool.map(func, lista)` — divide o iterável pelos processos disponíveis.

```
from multiprocessing import Pool

def quadrado(n):
    return n * n

with Pool() as pool:
    resultados = pool.map(quadrado, range(1, 11))
```

Process vs Pool

Característica	Process (Manual)	Pool (Abstração)
Controlo	Total sobre cada instância	Automatizado
Escalabilidade	Difícil com muitas tarefas	Excelente para processamento em lote
Caso de uso	Tarefas longas e independentes	Muitas tarefas curtas (Data Science)

2.7 Medição de Desempenho

```
import time
inicio = time.perf_counter()
# código a medir
fim = time.perf_counter()
duracao = fim - inicio
```

- `perf_counter()` — alta resolução, relógio monotónico.
- Uma única medição não chega → repetir 5–10 vezes e calcular média.

2.8 Arquitetura Híbrida (Threads + Processos)

Entrada I/O → THREADS (baixo overhead, GIL libertado durante rede/disco)
Cálculo CPU → PROCESSOS (contorna GIL, usa todos os núcleos)
Saída I/O → THREADS

MÓDULO 3 — Comunicação entre Processos (IPC)

3.1 O Problema: Memória Isolada

Processos **não partilham memória**. Se o Processo A alterar uma variável global, o Processo B **não vê a alteração**. Comunicação deve ser explícita.

3.2 Message Passing (Passagem de Mensagens)

- Dados enviados explicitamente de um processo para outro.
- Em Python, os objetos são serializados com **Pickle**: `pickle.dumps()` → transmissão → `pickle.loads()`.

3.3 `multiprocessing.Queue` ⚠

- Estrutura **FIFO** (First-In, First-Out).
- Thread/process-safe para múltiplos produtores e consumidores.
- Serializa automaticamente com Pickle.

Método	Descrição
<code>q.put(item)</code>	Adiciona à fila (bloqueia se cheia)
<code>q.get()</code>	Remove e retorna (bloqueia se vazia)
<code>q.empty()</code>	Verifica se vazia — NÃO FIÁVEL em concorrência!

Padrão Produtor-Consumidor com Sentinela

```
def produtor(q):
    for i in range(5):
        q.put(i)
    q.put(None) # sentinela — fim de dados

def consumidor(q):
    while True:
        item = q.get()
        if item is None:
            break
        print(item)
```

⚠ **NUNCA** usar `q.empty()` para decidir quando parar — race condition! **SEMPRE** usar sentinela `None` (um `None` por consumidor).

`queue.Queue` vs `multiprocessing.Queue`

	<code>queue.Queue</code>	<code>multiprocessing.Queue</code>
Para	Threads	Processos
Memória	Partilhada	Serializa via Pickle
Erro comum	—	Passar <code>queue.Queue</code> para <code>Process</code> causa erro (processos não partilham memória)

3.4 `multiprocessing.Pipe` ⚠

- Comunicação **ponto-a-ponto** entre dois processos.
- `Pipe()` devolve **dois endpoints**: `conn1, conn2 = Pipe()`.
- Cada processo usa **uma extremidade**.
- Mais rápido que Queue para comunicação 1-para-1; menos flexível com múltiplos intervenientes.

Método	Descrição
<code>conn.send(obj)</code>	Envia objeto para a outra extremidade
<code>conn.recv()</code>	Bloqueia até receber um objeto
<code>conn.close()</code>	Fecha a extremidade

```
from multiprocessing import Process, Pipe

def produtor(conn):
    conn.send({"nome": "Ana", "curso": "LEI"})
    conn.close()

def consumidor(conn):
    print(conn.recv())
    conn.close()

if __name__ == "__main__":
    conn1, conn2 = Pipe()
    p1 = Process(target=produtor, args=(conn1,))
    p2 = Process(target=consumidor, args=(conn2,))
    p1.start(); p2.start()
    p1.join(); p2.join()
```

3.5 Partilha de Estado: `Value` e `Array`

Para partilhar estado diretamente (sem copiar):

```
from multiprocessing import Value, Array, Lock

v = Value('i', 0)  # inteiro partilhado, valor inicial 0
a = Array('d', 10) # array de 10 doubles

# Acesso não é sincronizado automaticamente → usar Lock
lock = Lock()
```



```
with lock:
    v.value += 1
```

Código de tipo	Tipo
'i'	Inteiro
'd'	Double (float)

3.6 Partilha Complexa: **Manager**

- Para estruturas dinâmicas: **dict**, **list**, objetos personalizados.
- Cria um processo servidor que gere objetos partilhados; outros processos recebem **proxies**.
- Maior overhead que **Value** (cada acesso envolve comunicação com o servidor).

```
import multiprocessing
manager = multiprocessing.Manager()
d = manager.dict() # dicionário partilhado
```

3.7 Deadlock e Armadilha com **join()**

Deadlock: Processos bloqueados mutuamente, cada um à espera que o outro liberte um recurso.

Armadilha clássica:

```
# ERRADO — pode causar deadlock
p1.join() # ← bloqueia aqui se a Queue estiver cheia
p2.join() # ← nunca chega aqui

# A Queue fila3 só é consumida depois — mas p2 bloqueou antes
while True:
    item = fila3.get()
    ...
```

Solução: Consumir a Queue **antes** de fazer **join()** dos processos que escrevem nela.

MÓDULO 4 — Sistemas Distribuídos e Sockets

4.1 O que é um Sistema Distribuído?

"Uma coleção de computadores independentes que se comportam como um único sistema coeso para o utilizador." — Tanenbaum

- Cada máquina tem a sua **própria memória e processador**.
- Comunicação por **rede** (troca de mensagens).
- Aparência de **sistema único** para o utilizador.

Desafios:

- **Latência:** nanossegundos localmente vs milissegundos na rede.
- **Consistência:** garantir que todos os nós têm a mesma visão dos dados.
- **Tolerância a falhas:** falha de um nó não derruba o sistema.
- **Escalabilidade:** horizontal (mais máquinas) vs vertical (mais recursos numa máquina).

4.2 Arquiteturas de Sistemas Distribuídos

Cliente-Servidor

- **Modelo centralizado:** servidor dedicado fornece serviços; clientes solicitam.
- Comunicação **assimétrica:** cliente inicia, servidor responde.
- **O servidor fica à escuta** (`listen`, `accept`); o **cliente envia pedidos** (`connect`). ⚠
- Exemplos: web, e-mail, bases de dados.

Peer-to-Peer (P2P) ⚠

- **Todos os nós têm funções equivalentes** — podem ser clientes e servidores ao mesmo tempo.
- Descentralizado — sem servidor central.
- Alta tolerância a falhas (falha de um nó não compromete o sistema).
- Escalabilidade natural (mais utilizadores = mais recursos).
- Exemplos: BitTorrent, Blockchain.

Microserviços

- Aplicação construída como conjunto de serviços pequenos e independentes.
- Cada serviço tem uma única responsabilidade.
- Comunicação via HTTP/REST ou filas de mensagens.
- Diferentes serviços podem usar diferentes tecnologias.

4.3 Sockets TCP em Python ⚠

O que é um socket?

"Um ponto de extremidade de comunicação bidirecional numa rede."

Identificado por: **Endereço IP + Porta** (ex: `192.168.1.10:8080`).

TCP vs UDP

	TCP	UDP
Conexão	Orientado à conexão	Sem conexão
Entrega	Garante entrega e ordem	Sem garantia
Velocidade	Mais lento	Mais rápido
Uso	HTTP, FTP, e-mail, bases de dados	Streaming, jogos, DNS, VoIP

API de Sockets em Python

```
import socket

# Criar socket TCP
s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
# AF_INET = IPv4; SOCK_STREAM = TCP (SOCK_DGRAM = UDP)

# SERVIDOR:
s.bind((host, port)) # associa endereço
s.listen()           # fica à escuta
conn, addr = s.accept() # aceita ligação (bloqueia até cliente ligar)
data = conn.recv(1024) # recebe dados
conn.sendall(data)     # envia dados
conn.close()

# CLIENTE:
s.connect((host, port)) # liga ao servidor
s.sendall(msg.encode()) # envia dados
data = s.recv(1024)    # recebe dados
s.close()
```

Fluxo do Servidor: `socket()` → `bind()` → `listen()` → `accept()` → `recv/send()` → `close()`

Fluxo do Cliente: `socket()` → `connect()` → `send()` → `recv()` → `close()`

Codificação

```
# Enviar: String → Bytes
msg.encode("utf-8")

# Receber: Bytes → String
data.decode("utf-8")
```

Servidor Iterativo vs Concorrente

Tipo	Comportamento	Limitação
Iterativo	Atende um cliente de cada vez; outros esperam no backlog	Não serve múltiplos clientes em simultâneo
Concorrente	Cria uma thread por cliente	Múltiplos clientes simultaneamente

```
# Servidor concorrente (uma thread por cliente)
while True:
    conn, addr = servidor.accept()
    thread = threading.Thread(target=tratar_cliente, args=(conn, addr),
daemon=True)
    thread.start()
```

Threads daemon terminam automaticamente quando a thread principal termina.

4.4 RPC — Remote Procedure Call ⚠

Conceito

- Permite invocar uma função num servidor remoto **como se fosse local**.
- O cliente **não precisa de saber** os detalhes de implementação ou localização do servidor.
- A camada RPC trata: endereçamento, buffering, serialização (marshalling).

JSON-RPC 2.0 (protocolo usado nas aulas)

Pedido (cliente → servidor):

```
{
  "jsonrpc": "2.0",
  "method": "add",
  "params": [5, 7],
  "id": 1
}
```

Resposta (servidor → cliente):

```
{
  "jsonrpc": "2.0",
  "result": 12,
  "id": 1
}
```

Resposta de erro:

```
{
  "jsonrpc": "2.0",
  "error": "Método não suportado",
  "id": null
}
```

Fluxo de dados

```
Objeto Python → json.dumps().encode() → bytes no socket TCP
→ data.decode() + json.loads() → Objeto Python no destino
```

Dispatch Dinâmico

```
# Em vez de if/elif para cada função:
functions = {
  "add": add,
```

```
"multiply": multiply
}
result = functions[method>(*params)
```

RPC como chamada local (`__getattr__`)

```
def __getattr__(self, name):
    def method(*args, **kwargs):
        return self.invoke(name, args)
    return method

# Uso:
client = RPCClient("localhost", 8000)
resultado = client.soma(3, 7) # parece local, executa remotamente
```

MÓDULO 5 — HTTP e REST APIs

5.1 HTTP (HyperText Transfer Protocol)

- Protocolo de camada de aplicação (L7) sobre TCP/IP.
- **Stateless** (sem estado): cada pedido é independente; servidor não guarda memória do cliente.
- Ciclo: **Request** → **Response**.

Anatomia de um Pedido HTTP (Request)

```
GET /index.html HTTP/1.1      ← Linha de pedido (método, URI, versão)
Host: localhost:8000          ← Cabeçalhos
User-Agent: Mozilla/5.0
Accept: text/html

                               ← Linha vazia obrigatória
[corpo opcional]
```

Anatomia de uma Resposta HTTP (Response)

```
HTTP/1.1 200 OK               ← Linha de status
Content-Type: application/json ← Cabeçalhos
Content-Length: 42

{"status": "success"}          ← Corpo
```

5.2 Métodos HTTP e CRUD

Método	CRUD	Descrição	Idempotente?
GET	Read	Recupera recurso (não altera estado)	Sim (Seguro)
POST	Create	Cria novo recurso	Não
PUT	Update	Substitui recurso	Sim
DELETE	Delete	Remove recurso	Sim

5.3 Códigos de Estado (Status Codes)

Classe	Significado	Exemplos
1xx	Informação	—
2xx	Sucesso	200 OK, 201 Created, 204 No Content
3xx	Redireção	301 Moved Permanently
4xx	Erro do cliente	404 Not Found, 400 Bad Request
5xx	Erro do servidor	500 Internal Server Error

5.4 REST (Representational State Transfer)

- Estilo arquitetural para APIs web.
- Recursos identificados por URIs (ex: `/api/tasks/1`).
- Operações via métodos HTTP (GET, POST, PUT, DELETE).
- Comunicação via **JSON**.

```
GET    /api/tasks      → lista todas as tarefas
GET    /api/tasks/1   → uma tarefa específica
POST   /api/tasks      → cria nova tarefa
PUT    /api/tasks/1 → atualiza tarefa
DELETE /api/tasks/1   → remove tarefa
```

5.5 Flask (Framework Web Python)

Flask abstrai sockets, parsing de HTTP, headers e threading.

```
from flask import Flask, jsonify, request

app = Flask(__name__)

@app.route('/api/tasks', methods=['GET'])
def get_tasks():
    return jsonify(tasks), 200

@app.route('/api/tasks', methods=['POST'])
def add_task():
    data = request.get_json()
    tasks.append(data)
```

```
        return jsonify(data), 201

@app.route('/api/tasks/<int:id>', methods=['DELETE'])
def delete(id):
    # lógica aqui
    return '', 204
```

5.6 Hierarquia de Abstração

Sockets TCP → Protocolo HTTP → Flask/Libs → REST API
(bytes) (semântica) (automação) (arquitetura)

"Frameworks automatizam e escondem a complexidade do protocolo."

RESUMO RÁPIDO — Tabelas de Consulta

Quando usar o quê

Situação	Abordagem
Tarefas I/O-bound (rede, disco)	threading
Tarefas CPU-bound (cálculo)	multiprocessing
Muitas ligações I/O simultâneas	asyncio
Comunicação entre 2 processos (simples)	Pipe
Comunicação entre N processos	Queue
Partilha de valor simples	Value + Lock
Partilha de estruturas complexas	Manager
Paralelo real multi-core	multiprocessing.Pool

Sockets — Funções e para quê

Função	Quem usa	Para quê
bind()	Servidor	Associa endereço/porta
listen()	Servidor	Fica à escuta
accept()	Servidor	Aceita ligação de cliente
connect()	Cliente	Liga ao servidor
sendall()	Ambos	Envia dados
recv()	Ambos	Recebe dados
close()	Ambos	Fecha ligação

Arquiteturas de sistemas distribuídos

Arquitetura	Característica chave
Cliente-Servidor	Servidor à escuta, cliente pede; centralizado
P2P	Todos os nós são clientes E servidores; descentralizado
Microserviços	Serviços independentes, comunicação via API

Taxonomia de Flynn

SISD	1 instrução, 1 dado	CPU sequencial
SIMD	1 instrução, N dados	GPU, NumPy
MIMD	N instruções, N dados	multiprocessing

Resumo baseado nos slides CPD 2025/2026, IPS/EST Setúbal